

UniApp【萌宠案例】图片预览与懒加载实现方案

在萌宠案例中，图片是核心展示内容，优化图片交互与加载性能直接影响用户体验。本方案将为案例中的轮播图、萌宠列表图、详情页多图等场景，添加图片预览功能（支持多图切换、手势缩放）和懒加载优化（减少首屏加载压力），同时兼容小程序、App、H5多端特性，确保功能稳定与性能提升。

一、核心技术原理

1. 图片预览：基于UniApp原生API

UniApp提供 `uni.previewImage` 原生API，支持多图预览、手势缩放、保存图片等基础功能，无需引入第三方插件，多端兼容性极佳。核心参数如下：

- `current`：当前预览图片的索引（必填）。
- `urls`：预览图片的URL数组（必填，支持本地路径与网络路径）。
- `longPressActions`：长按图片的自定义操作（如保存、转发，可选）。



注意：预览图片的URL数组需与页面展示图片的URL完全一致，确保索引匹配；本地图片需使用绝对路径（如 `/static/pet/cat1.jpg`），网络图片需确保跨域配置正常。

2. 图片懒加载：UniApp图片组件原生支持

UniApp的 `<image>` 组件内置 `lazy-load` 属性，启用后图片会在进入视口区域时才开始加载，有效减少首屏请求数量，提升页面加载速度。核心特性：

- 默认支持小程序、App（iOS/Android）、H5端，无需额外配置。
- 配合 `mode` 属性（如 `widthFix`）使用，避免图片加载时布局抖动。
- 可通过 `placeholder-image` 属性设置占位图，优化加载过渡体验。

二、全局配置：基础样式与工具类

先定义图片懒加载的占位图样式与图片预览的通用工具方法，确保多页面样式统一与代码复用。

1. 全局样式 (App.vue)

Code block

```
1  /* 图片占位图通用样式 */
2  .image-placeholder {
3    background-color: #F5F5F5;
4    display: flex;
5    align-items: center;
6    justify-content: center;
7  }
8  /* 图片加载失败样式 */
9  .image-error {
10   background-color: #F5F5F5;
11   display: flex;
12   align-items: center;
13   justify-content: center;
14 }
15 .image-error::after {
16   content: "图片加载失败";
17   font-size: 24rpx;
18   color: #999;
19 }
20
21 /* 通用圆角样式 */
22 :root {
23   --img-radius: 16rpx; /* 图片统一圆角 */
24 }
25
```

2. 图片工具类 (utils/imageUtil.js)

封装图片预览方法，支持传入当前图片索引与URL数组，统一处理长按事件。

Code block

```
1  /**
2   * 图片工具类
3   * 包含图片预览、图片错误处理等通用方法
4   */
5  export const imageUtil = {
6    /**
7     * 图片预览
8     * @param {Object} options - 预览配置
9     * @param {number} options.current - 当前预览图片索引
10    * @param {Array} options.urls - 预览图片URL数组

```

```

11  * @param {string} options.title - 预览标题 (可选)
12  */
13  previewImage: (options) => {
14      const { current, urls, title = '萌宠图片' } = options
15      if (!urls || urls.length === 0) {
16          uni.showToast({ title: '无预览图片', icon: 'none' })
17          return
18      }
19      // 调用UniApp原生预览API
20      uni.previewImage({
21          current,
22          urls,
23          title,
24          // 长按图片自定义操作
25          longPressActions: {
26              itemList: ['保存图片', '转发图片'],
27              success: (res) => {
28                  const tapIndex = res.tapIndex
29                  if (tapIndex === 0) {
30                      // 保存图片到相册
31                      uni.saveImageToPhotosAlbum({
32                          filePath: urls[current],
33                          success: () => {
34                              uni.showToast({ title: '图片保存成功' })
35                          },
36                          fail: (err) => {
37                              console.error('图片保存失败: ', err)
38                              if (err.errMsg.includes('auth deny')) {
39                                  uni.showToast({ title: '请开启相册权限', icon: 'none' })
40                              } else {
41                                  uni.showToast({ title: '图片保存失败', icon: 'error' })
42                              }
43                          }
44                      })
45                  } else if (tapIndex === 1) {
46                      // 转发图片 (仅小程序/App支持)
47                      uni.share({
48                          title: title,
49                          imageUrl: urls[current],
50                          success: () => {
51                              uni.showToast({ title: '转发成功' })
52                          }
53                      })
54                  }
55              }
56          }
57      })

```

```
58     },
59
60     /**
61     * 图片加载错误处理
62     * @param {Object} e - 图片错误事件对象
63     */
64     handleImageError: (e) => {
65         // 加载失败时替换为默认图片
66         const imgElement = e.target
67         imgElement.src = '/static/default-pet.png'
68     }
69 }
70
```

三、页面改造：分场景实现图片功能

针对萌宠案例的核心页面（首页、分类页、详情页），分别实现图片懒加载与预览功能，适配不同的图片展示场景。

1. 首页改造：轮播图+推荐列表图

首页包含轮播图（多图）与萌宠推荐列表图（单图），为轮播图添加多图预览，为列表图添加单图预览与懒加载。

1.1 页面结构（pages/index/index.vue）

Code block

```
1  <template>
2    <view class="index-page">
3
4      <view class="swiper-container" v-if="swiperList.length">
5        <swiper indicator-dots autoplay circular interval="3000" duration="500">
6          <swiper-item v-for="(item, index) in swiperList" :key="item.id">
7            <image
8              :src="item.imgUrl"
9              mode="widthFix"
10             class="swiper-img"
11             lazy-load
12             placeholder-image="/static/placeholder.png"
13             @tap="handleSwiperPreview(index)"
14             @error="imageUtil.handleImageError"
15           ></image>
16          </swiper-item>
17        </swiper>
18      </view>
```

```

19
20
21     <view class="pet-recommend" v-if="recommendList.length">
22         <view class="section-header">
23             <text class="header-title">萌宠推荐</text>
24             <navigator url="/pages/category/category" class="more-link">查看更多
</navigator>
25         </view>
26         <view class="recommend-list">
27             <navigator
28                 class="pet-item pet-card"
29                 v-for="(pet, petIndex) in recommendList"
30                 :key="pet.id"
31                 :url="'/pages/detail/detail?id='+pet.id"
32             >
33                 <image
34                     :src="pet.coverImg"
35                     mode="widthFix"
36                     class="pet-img"
37                     lazy-load
38                     placeholder-image="/static/placeholder.png"
39                     @tap.stop="handlePetImgPreview(petIndex)"
40                     @error="imageUtil.handleImageError"
41                 ></image>
42                 <view class="pet-info">
43                     <view class="pet-name">{{ pet.name }}</view>
44                     <view class="pet-breed">品种: {{ pet.breed }}</view>
45                     <view class="pet-meta">
46                         <view class="like-count"><image src="/static/icon/like.png"
mode="widthFix" class="like-icon"></image> {{ pet.likeCount }}</view>
47                         <view class="view-count">浏览 {{ pet.viewCount }}</view>
48                     </view>
49                     </navigator>
50                 </view>
51             </view>
52         </view>
53     </view>
54 </template>
55

```

1.2 逻辑实现 (pages/index/index.vue)

Code block

```

1  <script setup>
2  import { ref, onLoad, computed } from 'vue'

```

```
3 import request from '../utils/request'
4 import { imageUtil } from '../utils/imageUtil'
5
6 // 页面数据
7 const isLoading = ref(true)
8 const swiperList = ref([]) // 轮播图数据
9 const recommendList = ref([]) // 推荐列表数据
10
11 // 计算属性：提取轮播图URL数组（用于预览）
12 const swiperImgUrls = computed(() => {
13   return swiperList.value.map(item => item.imgUrl)
14 })
15
16 // 计算属性：提取推荐列表图片URL数组（用于预览）
17 const petImgUrls = computed(() => {
18   return recommendList.value.map(item => item.coverImg)
19 })
20
21 // 页面加载：获取数据
22 onLoad(async () => {
23   try {
24     const homeData = await request.get('/home')
25     swiperList.value = homeData.swiper || []
26     recommendList.value = homeData.recommend || []
27   } catch (err) {
28     console.error('首页数据加载失败：', err)
29   } finally {
30     isLoading.value = false
31   }
32 })
33
34 // 轮播图图片预览
35 const handleSwiperPreview = (index) => {
36   imageUtil.previewImage({
37     current: index,
38     urls: swiperImgUrls.value,
39     title: '热门萌宠轮播图'
40   })
41 }
42
43 // 萌宠列表图片预览
44 const handlePetImgPreview = (index) => {
45   imageUtil.previewImage({
46     current: index,
47     urls: petImgUrls.value,
48     title: recommendList.value[index].name
49   })
50 }
```

```
50 }
51 </script>
52
```

1.3 样式补充 (pages/index/index.vue)

Code block

```
1  /* 轮播图样式 */
2  .swiper-container {
3    width: 100%;
4    height: 300rpx;
5    margin-bottom: 20rpx;
6    padding: 0 24rpx;
7  }
8  .swiper-img {
9    width: 100%;
10   height: 100%;
11   border-radius: var(--img-radius);
12 }
13
14 /* 萌宠推荐图片样式 */
15 .pet-img {
16   width: 100%;
17   height: 300rpx;
18   border-radius: var(--img-radius);
19   margin-bottom: 16rpx;
20 }
21
22 /* 占位图与错误图适配 */
23 image {
24   &[lazy-load] {
25     background-color: #F5F5F5;
26   }
27 }
28
```

2. 分类页改造：列表图优化

分类页为长列表展示，重点为列表图添加懒加载，减少首屏加载压力，同时支持单图预览。

2.1 页面结构与逻辑 (pages/category/category.vue)

Code block

```
1  <template>
2    <view class="category-page">
3
4      <view class="category-tabs"></view>
5
6
7      <scroll-view
8        class="category-list"
9        scroll-y
10       refresher-enabled
11       @refresherrefresh="onRefresh"
12       @scrolltolower="onLoadMore"
13     >
14       <view class="pet-item pet-card" v-for="(pet, index) in petList"
15       :key="pet.id">
16         <navigator :url="'/pages/detail/detail?id='+pet.id" class="pet-link">
17           <image
18             :src="pet.coverImg"
19             mode="widthFix"
20             class="pet-img"
21             lazy-load
22             placeholder-image="/static/placeholder.png"
23             @tap.stop="handleImgPreview(index)"
24             @error="imageUtil.handleImageError"
25           ></image>
26           <view class="pet-info"></view>
27         </navigator>
28       </view>
29     </scroll-view>
30   </view>
31 </template>
32
33 <script setup>
34 import { ref, computed } from 'vue'
35 import request from '../utils/request'
36 import { imageUtil } from '../utils/imageUtil'
37
38 // 页面数据（省略部分原有数据）
39 const petList = ref([])
40
41 // 计算属性：提取列表图片URL数组
42 const petImgUrls = computed(() => {
43   return petList.value.map(item => item.coverImg)
44 })
45
46 // 图片预览
47 const handleImgPreview = (index) => {
```



```

47     imageUtil.previewImage({
48       current: index,
49       urls: petImgUrls.value,
50       title: petList.value[index].name
51     })
52   }
53
54   // 原有分页加载逻辑（onRefresh、onLoadMore等）保持不变
55 </script>
56
57 <style lang="scss" scoped>
58   /* 列表图片样式 */
59   .pet-img {
60     width: 100%;
61     height: 240rpx;
62     border-radius: var(--img-radius);
63     margin-bottom: 16rpx;
64   }
65 </style>
66

```

3. 详情页改造：多图预览核心场景

详情页顶部为萌宠多图展示，是图片预览的核心场景，需支持多图切换预览，同时为相关推荐列表图添加懒加载与预览。

3.1 页面结构（pages/detail/detail.vue）

Code block

```

1   <template>
2     <view class="detail-page">
3
4       <view class="pet-banner" v-if="petImgList.length">
5         <swiper indicator-dots circular autoplay interval="5000">
6           <swiper-item v-for="(img, index) in petImgList" :key="index">
7             <image
8               :src="img"
9               mode="widthFix"
10              class="banner-img"
11              lazy-load
12              placeholder-image="/static/placeholder.png"
13              @tap="handleBannerPreview(index)"
14              @error="imageUtil.handleImageError"
15            ></image>
16          </swiper-item>

```

```

17         </swiper>
18
19         <view class="img-count">{{ currentBannerIndex + 1 }}/{{
petImgList.length }}</view>
20     </view>
21
22
23     <view class="pet-base-info"></view>
24
25
26     <view class="related-recommend" v-if="relatedList.length">
27         <view class="section-title">相关推荐</view>
28         <view class="related-list">
29             <navigator
30                 class="related-item pet-card"
31                 v-for="(pet, index) in relatedList"
32                 :key="pet.id"
33                 :url="'/pages/detail/detail?id='+pet.id"
34             >
35                 <image
36                     :src="pet.coverImg"
37                     mode="widthFix"
38                     class="related-img"
39                     lazy-load
40                     placeholder-image="/static/placeholder.png"
41                     @tap.stop="handleRelatedPreview(index)"
42                     @error="imageUtil.handleImageError"
43                 ></image>
44                 <view class="related-name">{{ pet.name }}</view>
45                 <view class="related-breed">{{ pet.breed }}</view>
46             </navigator>
47         </view>
48     </view>
49 </view>
50 </template>
51

```

3.2 逻辑实现 (pages/detail/detail.vue)

Code block

```

1  <script setup>
2  import { ref, onLoad, computed } from 'vue'
3  import request from '../utils/request'
4  import { imageUtil } from '../utils/imageUtil'
5

```

```
6 // 页面数据
7 const petId = ref('')
8 const petImgList = ref([]) // 顶部多图URL数组
9 const relatedList = ref([]) // 相关推荐数据
10 const currentBannerIndex = ref(0) // 当前轮播图索引
11
12 // 计算属性：提取相关推荐图片URL数组
13 const relatedImgUrls = computed(() => {
14   return relatedList.value.map(item => item.coverImg)
15 })
16
17 // 页面加载：获取详情数据
18 onLoad(async (options) => {
19   if (!options.id) {
20     uni.navigateBack()
21     return
22   }
23   petId.value = options.id
24   await getPetDetail()
25 })
26
27 // 获取萌宠详情（原有逻辑）
28 const getPetDetail = async () => {
29   try {
30     const data = await request.get(`/pet/${petId.value}`)
31     petImgList.value = data.petImgList || []
32     relatedList.value = data.relatedList || []
33   } catch (err) {
34     console.error('详情数据加载失败：', err)
35   }
36 }
37
38 // 顶部轮播图预览
39 const handleBannerPreview = (index) => {
40   imageUtil.previewImage({
41     current: index,
42     urls: petImgList.value,
43     title: petInfo.value.name
44   })
45 }
46
47 // 监听轮播图切换，更新当前索引
48 const handleBannerChange = (e) => {
49   currentBannerIndex.value = e.detail.current
50 }
51
52 // 相关推荐图片预览
```

```
53 const handleRelatedPreview = (index) => {
54   imageUtil.previewImage({
55     current: index,
56     urls: relatedImgUrls.value,
57     title: relatedList.value[index].name
58   })
59 }
60 </script>
61
```

3.3 样式补充 (pages/detail/detail.vue)

Code block

```
1  /* 顶部多图样式 */
2  .pet-banner {
3    position: relative;
4    width: 100%;
5    height: 400rpx;
6  }
7  .banner-img {
8    width: 100%;
9    height: 100%;
10 }
11 /* 图片数量提示 */
12 .img-count {
13   position: absolute;
14   bottom: 20rpx;
15   right: 24rpx;
16   background-color: rgba(0, 0, 0, 0.5);
17   color: #fff;
18   font-size: 24rpx;
19   padding: 4rpx 16rpx;
20   border-radius: 16rpx;
21   z-index: 10;
22 }
23
24 /* 相关推荐图片样式 */
25 .related-img {
26   width: 100%;
27   height: 200rpx;
28   border-radius: var(--img-radius);
29   margin-bottom: 12rpx;
30 }
31
```

四、多端适配与性能优化要点

1. 多端适配注意事项

- **小程序端：**
 图片懒加载在微信小程序中需确保基础库版本≥2.18.0。
- 保存图片到相册需提前获取用户授权，可通过 `uni.getSetting` 检查权限状态。

App端：

Android端需在 `manifest.json` 中配置相册权限
(`android.permission.WRITE_EXTERNAL_STORAGE`) 。

iOS端需在 `info.plist` 中添加 `NSPhotoLibraryAddUsageDescription` （保存图片描述）。

H5端：

图片懒加载依赖浏览器的Intersection Observer API，低版本浏览器（如IE）不支持，可通过引入polyfill兼容。

H5端无原生分享功能，需在 `imageUtil` 中判断平台，H5端隐藏转发选项。

2. 性能优化技巧

- **占位图优化：**使用与目标图片尺寸一致的占位图（如纯色图），避免图片加载时布局抖动。
- **图片尺寸优化：**服务端返回图片时，根据设备尺寸返回合适分辨率的图片（如小程序端返回750px宽的图片），减少图片体积。
- **预加载策略：**详情页顶部轮播图可预加载下一张图片，提升切换流畅度，通过 `new Image().src = 下一张图片URL` 实现。
- **列表性能：**分类页长列表使用 `scroll-view` 配合懒加载，同时为列表项添加 `v-memo` 优化渲染性能。

3. 常见问题解决

问题现象	原因分析	解决方案
图片预览时索引不匹配	预览的URL数组与页面展示的图片URL不一致	确保预览用的urls数组是直接 从页面数据中提取的，避免二 次处理导致顺序变化
懒加载不生效	未添加 <code>lazy-load</code> 属性；图 片已在首屏内；H5端低版本 浏览器不支持	1. 确认添加 <code>lazy-load</code> 属 性；2. 首屏外图片才会触发懒 加载；3. H5端引入

		Intersection Observer polyfill
保存图片失败	无相册权限；图片路径错误； 跨域图片无法保存	1. 引导用户开启权限；2. 检查图片路径是否正确；3. 网络图片需服务端配置CORS允许保存

五、总结

本方案通过UniApp原生API与组件特性，为萌宠案例实现了完整的图片预览与懒加载功能，核心价值如下：

- 1. **交互体验提升**：图片预览支持多图切换、手势缩放、长按保存/转发，满足用户核心交互需求。
- 2. **性能优化显著**：懒加载减少首屏图片请求数量，降低页面加载时间，提升弱网环境下的用户体验。
- 3. **多端兼容性好**：基于UniApp原生能力开发，无需第三方插件，适配小程序、App、H5全平台。
- 4. **代码复用性高**：封装图片工具类，统一处理预览与错误逻辑，减少页面重复代码。

实际开发中，可根据业务需求扩展功能，如添加图片压缩上传（针对用户上传场景）、图片水印（版权保护）等，进一步完善萌宠案例的图片相关能力。

（注：文档部分内容可能由 AI 生成）